Lesson 4 Assignment

Data Processing - Python Development and Pandas

Course: Data Analytics / Python Development

This notebook completes all required Lesson 4 tasks. Each task includes clear comments, output printing, simple validation, and error handling where appropriate.

```
# Import required libraries
# pandas and numpy are used for data processing.
# reduce is used for the reduce() task.

import pandas as pd
import numpy as np
from functools import reduce

print("Libraries imported successfully.")
```

```
Libraries imported successfully.
```

Task 1: Create a DataFrame

Create a DataFrame with Name, Age, and City columns. Check that the column names and data types are correct.

```
try:
    # Create the DataFrame using a dictionary.
    people_data = {
        "Name": ["Alice", "Bob", "Charlie", "David"],
        "Age": [25, 30, 35, 28],
        "City": ["New York", "San Francisco", "Los Angeles", "Chicago"]
    }

    df_people = pd.DataFrame(people_data)

    # Basic validation to make sure the required columns exist.
    expected_columns = ["Name", "Age", "City"]
    if list(df_people.columns) != expected_columns:
        raise ValueError("The DataFrame columns are not correct.")

    print("DataFrame created successfully:")
    print(df_people)
```

```
    print("\nData types:")
    print(df_people.dtypes)

except Exception as error:
    print(f"An error occurred while creating the DataFrame: {error}")
```

```
DataFrame created successfully:
      Name  Age           City
0    Alice   25       New York
1      Bob   30  San Francisco
2  Charlie   35    Los Angeles
3    David   28        Chicago

Data types:
Name    object
Age      int64
City    object
dtype: object
```

Task 2: Row and Column Manipulation

Drop the City column and verify that only Name and Age remain.

```
try:
    # Drop the City column. errors='raise' helps catch mistakes if the column c
    df_name_age = df_people.drop(columns=["City"], errors="raise")

    # Verify the result.
    if list(df_name_age.columns) == ["Name", "Age"]:
        print("City column dropped successfully.")
    else:
        raise ValueError("The resulting DataFrame does not contain only Name ar

    print(df_name_age)

except KeyError:
    print("Error: The 'City' column was not found in the DataFrame.")
except Exception as error:
    print(f"An error occurred while dropping the column: {error}")
```

```
City column dropped successfully.
      Name  Age
0    Alice   25
1      Bob   30
2  Charlie   35
3    David   28
```

Task 3: Handling Null Values

Create a DataFrame with null values in each column and fill them using appropriate values.

```
try:
    # Create a DataFrame with at least one missing value in each column.
    df_missing = pd.DataFrame({
        "Name": ["Sara", None, "Mike", "Nina"],
        "Age": [22, np.nan, 31, 29],
        "City": ["Toronto", "Ottawa", None, "Mississauga"]
    })

    print("Original DataFrame with missing values:")
    print(df_missing)
    print("\nMissing values before filling:")
    print(df_missing.isnull().sum())

    # Fill missing values based on column type.
    # Text columns are filled with 'Unknown'. Numeric Age is filled with the av
    average_age = df_missing["Age"].mean()
    df_filled = df_missing.copy()
    df_filled["Name"] = df_filled["Name"].fillna("Unknown")
    df_filled["Age"] = df_filled["Age"].fillna(average_age)
    df_filled["City"] = df_filled["City"].fillna("Unknown")

    print("\nDataFrame after filling missing values:")
    print(df_filled)
    print("\nMissing values after filling:")
    print(df_filled.isnull().sum())

except Exception as error:
    print(f"An error occurred while handling null values: {error}")
```

```
Original DataFrame with missing values:
   Name   Age         City
0  Sara  22.0      Toronto
1  None   NaN       Ottawa
2  Mike  31.0         None
3  Nina  29.0  Mississauga

Missing values before filling:
Name    1
Age     1
City    1
dtype: int64

DataFrame after filling missing values:
      Name        Age         City
0     Sara  22.000000      Toronto
1  Unknown  27.333333       Ottawa
2     Mike  31.000000      Unknown
3     Nina  29.000000  Mississauga
```

```
Missing values after filling:
Name    0
Age     0
City    0
dtype: int64
```

Task 4: GroupBy and Describe

Group by Category and apply describe() to the Value column. Then interpret the statistics.

```
try:
    df_group = pd.DataFrame({
        "Category": ["A", "B", "A", "B", "A", "C"],
        "Value": [10, 20, 15, 25, 30, 35]
    })

    group_summary = df_group.groupby("Category")["Value"].describe()

    print("Original DataFrame:")
    print(df_group)
    print("\nDescriptive statistics by category:")
    print(group_summary)

    print("\nInterpretation:")
    print("Category A has 3 records with an average value of 18.33.")
    print("Category B has 2 records with an average value of 22.50.")
    print("Category C has 1 record, so its standard deviation is NaN because or

except Exception as error:
    print(f"An error occurred during GroupBy and describe: {error}")
```

```
Original DataFrame:
  Category  Value
0        A     10
1        B     20
2        A     15
3        B     25
4        A     30
5        C     35

Descriptive statistics by category:
          count       mean        std   min    25%   50%    75%   max
Category
A           3.0  18.333333  10.408330  10.0  12.50  15.0  22.50  30.0
B           2.0  22.500000   3.535534  20.0  21.25  22.5  23.75  25.0
C           1.0  35.000000        NaN  35.0  35.00  35.0  35.00  35.0

Interpretation:
Category A has 3 records with an average value of 18.33.
```

```
Category B has 2 records with an average value of 22.50.
Category C has 1 record, so its standard deviation is NaN because only one value
```

Task 5: Concatenation and Merging

Concatenate df1 and df2 vertically, then merge the result with df3 horizontally while preserving all rows.

```
try:
    df1 = pd.DataFrame({"A": [1, 2], "B": [3, 4]})
    df2 = pd.DataFrame({"A": [5, 6], "B": [7, 8]})
    df3 = pd.DataFrame({"C": [9, 10], "D": [11, 12]})

    # Step 1: Concatenate vertically.
    vertical_concat = pd.concat([df1, df2], ignore_index=True)

    # Step 2: Create a matching key so df3 can be merged horizontally.
    # df3 has 2 rows, so the key repeats to match the 4 rows in vertical_concat
    vertical_concat["Merge_Key"] = [0, 1, 0, 1]
    df3_with_key = df3.copy()
    df3_with_key["Merge_Key"] = [0, 1]

    merged_result = pd.merge(vertical_concat, df3_with_key, on="Merge_Key", how
    merged_result = merged_result.drop(columns=["Merge_Key"])

    print("df1:")
    print(df1)
    print("\ndf2:")
    print(df2)
    print("\ndf3:")
    print(df3)
    print("\nAfter vertical concatenation of df1 and df2:")
    print(vertical_concat.drop(columns=["Merge_Key"]))
    print("\nFinal horizontally merged result:")
    print(merged_result)
    print(f"\nFinal shape: {merged_result.shape}")

    if merged_result.shape != (4, 4):
        raise ValueError("The final DataFrame shape is not correct.")

except Exception as error:
    print(f"An error occurred during concatenation or merging: {error}")
```

```
df1:
   A  B
0  1  3
1  2  4
```

```
df2:
   A  B
0  5  7
1  6  8

df3:
    C   D
0   9  11
1  10  12

After vertical concatenation of df1 and df2:
   A  B
0  1  3
1  2  4
2  5  7
3  6  8

Final horizontally merged result:
   A  B   C   D
0  1  3   9  11
1  2  4  10  12
2  5  7   9  11
3  6  8  10  12

Final shape: (4, 4)
```

Task 6: Tuples and Sets

Create a tuple and a set. Attempt to add an element to each and explain the difference.

```
# A tuple is immutable, so it cannot be changed using an add() method.
fruits_tuple = ("apple", "banana", "mango")
numbers_set = {1, 2, 3, 4, 5}

print("Original tuple:", fruits_tuple)
print("Original set:", numbers_set)

try:
    fruits_tuple.add("orange")
except AttributeError as error:
    print("\nTuple add attempt result:")
    print("Tuples are immutable and do not have an add() method.")
    print("Error message:", error)

try:
    numbers_set.add(6)
    print("\nSet after adding 6:", numbers_set)
except Exception as error:
    print(f"An error occurred while adding to the set: {error}")
```

```
print("\nExplanation:")
print("A tuple cannot be modified after creation. A set is mutable, so new unic
```

```
Original tuple: ('apple', 'banana', 'mango')
Original set: {1, 2, 3, 4, 5}

Tuple add attempt result:
Tuples are immutable and do not have an add() method.
Error message: 'tuple' object has no attribute 'add'

Set after adding 6: {1, 2, 3, 4, 5, 6}

Explanation:
A tuple cannot be modified after creation. A set is mutable, so new unique eleme
```

Task 7: Dictionaries

Create a student score dictionary, update an existing score, and add a new student.

```
try:
    student_scores = {
        "Ali": 85,
        "Sara": 90,
        "Ahmed": 78
    }

    print("Original dictionary:")
    print(student_scores)

    # Update an existing student's score.
    student_scores["Ali"] = 92

    # Add a new student.
    student_scores["Fatima"] = 88

    print("\nUpdated dictionary:")
    print(student_scores)

except Exception as error:
    print(f"An error occurred while working with dictionaries: {error}")
```

```
Original dictionary:
{'Ali': 85, 'Sara': 90, 'Ahmed': 78}

Updated dictionary:
{'Ali': 92, 'Sara': 90, 'Ahmed': 78, 'Fatima': 88}
```

Task 8: Functions and Lambda

Define a regular function and a lambda function to calculate the square of a number.

```
def square_regular(number):
    """Return the square of a number."""
    return number ** 2

square_lambda = lambda number: number ** 2

try:
    test_numbers = [4, 7]

    for value in test_numbers:
        print(f"Regular function: square of {value} = {square_regular(value)}")
        print(f"Lambda function: square of {value} = {square_lambda(value)}")

except TypeError:
    print("Error: Please provide numeric input only.")
except Exception as error:
    print(f"An error occurred while testing functions: {error}")
```

```
Regular function: square of 4 = 16
Lambda function: square of 4 = 16
Regular function: square of 7 = 49
Lambda function: square of 7 = 49
```

Task 9: Iterators and Generators

Create an iterator class and a generator function that both produce the first 5 even numbers.

```
class FirstFiveEvenNumbers:
    """Iterator class that generates the first five even numbers: 2, 4, 6, 8, 1

    def __init__(self):
        self.current_number = 0
        self.count = 0
        self.limit = 5

    def __iter__(self):
        return self

    def __next__(self):
        if self.count < self.limit:
            self.current_number += 2
```

```
            self.count += 1
            return self.current_number
        raise StopIteration

def even_number_generator():
    """Generator function that yields the first five even numbers."""
    for number in range(2, 12, 2):
        yield number

try:
    iterator_output = list(FirstFiveEvenNumbers())
    generator_output = list(even_number_generator())

    print("Iterator output:", iterator_output)
    print("Generator output:", generator_output)

except Exception as error:
    print(f"An error occurred while creating iterator or generator: {error}")
```

```
Iterator output: [2, 4, 6, 8, 10]
Generator output: [2, 4, 6, 8, 10]
```

Task 10: Map, Reduce, and Filter

Use map() to square numbers, reduce() to find the product, and filter() to get even numbers.

```
try:
    numbers = [1, 2, 3, 4, 5]

    squared_numbers = list(map(lambda number: number ** 2, numbers))
    product_of_numbers = reduce(lambda x, y: x * y, numbers)
    even_numbers = list(filter(lambda number: number % 2 == 0, numbers))

    print("Original numbers:", numbers)
    print("Squared numbers using map():", squared_numbers)
    print("Product using reduce():", product_of_numbers)
    print("Even numbers using filter():", even_numbers)

except TypeError:
    print("Error: The list must contain numbers only.")
except Exception as error:
    print(f"An error occurred while using map, reduce, or filter: {error}")
```

```
Original numbers: [1, 2, 3, 4, 5]
Squared numbers using map(): [1, 4, 9, 16, 25]
Product using reduce(): 120
Even numbers using filter(): [2, 4]
```

Task 11: Object-Oriented Programming

Create a Rectangle class with length and width. Add methods for area and perimeter.

```
class Rectangle:
    """A class used to represent a rectangle."""

    def __init__(self, length, width):
        if length <= 0 or width <= 0:
            raise ValueError("Length and width must be positive numbers.")
        self.length = length
        self.width = width

    def calculate_area(self):
        """Calculate and return the area of the rectangle."""
        return self.length * self.width

    def calculate_perimeter(self):
        """Calculate and return the perimeter of the rectangle."""
        return 2 * (self.length + self.width)

try:
    rectangle_1 = Rectangle(5, 3)
    rectangle_2 = Rectangle(10, 4)

    rectangles = [rectangle_1, rectangle_2]

    for index, rectangle in enumerate(rectangles, start=1):
        print(f"Rectangle {index}: length = {rectangle.length}, width = {rectar
        print(f"Area = {rectangle.calculate_area()}")
        print(f"Perimeter = {rectangle.calculate_perimeter()}\n")

except ValueError as error:
    print(f"Invalid rectangle dimensions: {error}")
except Exception as error:
    print(f"An error occurred while working with the Rectangle class: {error}")
```

```
Rectangle 1: length = 5, width = 3
Area = 15
Perimeter = 16

Rectangle 2: length = 10, width = 4
Area = 40
Perimeter = 28
```

Task 12: Pandas Data Analysis

Calculate average salary by department, display employees with salary above 60000, and add a Bonus column.

```
try:
    df_employees = pd.DataFrame({
        "Name": ["John", "Jane", "Bob", "Alice", "Charlie"],
        "Department": ["IT", "HR", "IT", "Finance", "HR"],
        "Salary": [55000, 65000, 70000, 60000, 58000]
    })

    # Check that the Salary column is numeric before doing calculations.
    if not pd.api.types.is_numeric_dtype(df_employees["Salary"]):
        raise TypeError("Salary column must be numeric.")

    average_salary_by_department = df_employees.groupby("Department")["Salary"]
    high_salary_employees = df_employees.loc[df_employees["Salary"] > 60000, "N
    df_employees["Bonus"] = df_employees["Salary"] * 0.10

    print("Employee DataFrame with Bonus column:")
    print(df_employees)
    print("\nAverage salary by department:")
    print(average_salary_by_department)
    print("\nEmployees with salary greater than 60000:")
    print(list(high_salary_employees))

except TypeError as error:
    print(f"Data type error: {error}")
except Exception as error:
    print(f"An error occurred during Pandas data analysis: {error}")
```

```
Employee DataFrame with Bonus column:
      Name Department  Salary   Bonus
0     John         IT   55000  5500.0
1     Jane         HR   65000  6500.0
2      Bob         IT   70000  7000.0
3    Alice    Finance   60000  6000.0
4  Charlie         HR   58000  5800.0

Average salary by department:
Department
Finance    60000.0
HR         61500.0
IT         62500.0
Name: Salary, dtype: float64

Employees with salary greater than 60000:
['Jane', 'Bob']
```

Final Submission Notes

- This notebook has been completed with outputs for every task.
- The code uses meaningful variable names, comments, validation checks, and try-except blocks.
- Submit this notebook as a PDF with outputs.
- Upload the notebook to Google Colab.

Colab Notebook Link:
https://colab.research.google.com/drive/1sJj0DYZwwYLzwGV_vhlOFtKvQ4iPLu5N?usp=sharing